

Relatório Técnico - Assistente Corporativo Stealth (WhatsApp + IA)

Projeto: noter.donadio - Pipeline Inteligente de Contatos e Negociações

Autor: Ruas.dev - Irving G. Ruas Lopes

Data: 20 de Julho de 2026

Versão: 1.0 (MVP)

Cliente: Caio Donadio

1. Sumário Executivo

Este documento detalha a arquitetura técnica para o desenvolvimento do MVP (Minimum Viable Product) do **noter.donadio**, um assistente corporativo stealth que integra WhatsApp com Inteligência Artificial. O sistema foi projetado sob encomenda para **Caio Donadio**, visando otimizar a gestão de contatos e negociações através da leitura passiva de mensagens, estruturação inteligente de informações e um pipeline visual totalmente gerenciável.

Stack Tecnológica Definida:

- Backend:** Node.js com TypeScript (Fastify)
- Frontend:** React PWA com Capacitor (mobile/desktop nativo)
- Conexão WhatsApp:** Baileys (WhiskeySockets) - Modo Multi-Device
- Banco de Dados:** PostgreSQL 16 + Redis 7
- Processamento Assíncrono:** BullMQ (Filas com Redis)
- IA:** OpenAI API (GPT-4o / GPT-4o-mini) + Whisper API (Transcrição)
- Infraestrutura:** Docker Compose (VPS 4GB RAM)

2. Visão Geral da Arquitetura

2.1 Padrão Arquitetural

O sistema adota uma arquitetura **Event-Driven (Orientada a Eventos)** combinada com o padrão **Offline-First com Cache**, organizada em microsserviços lógicos:

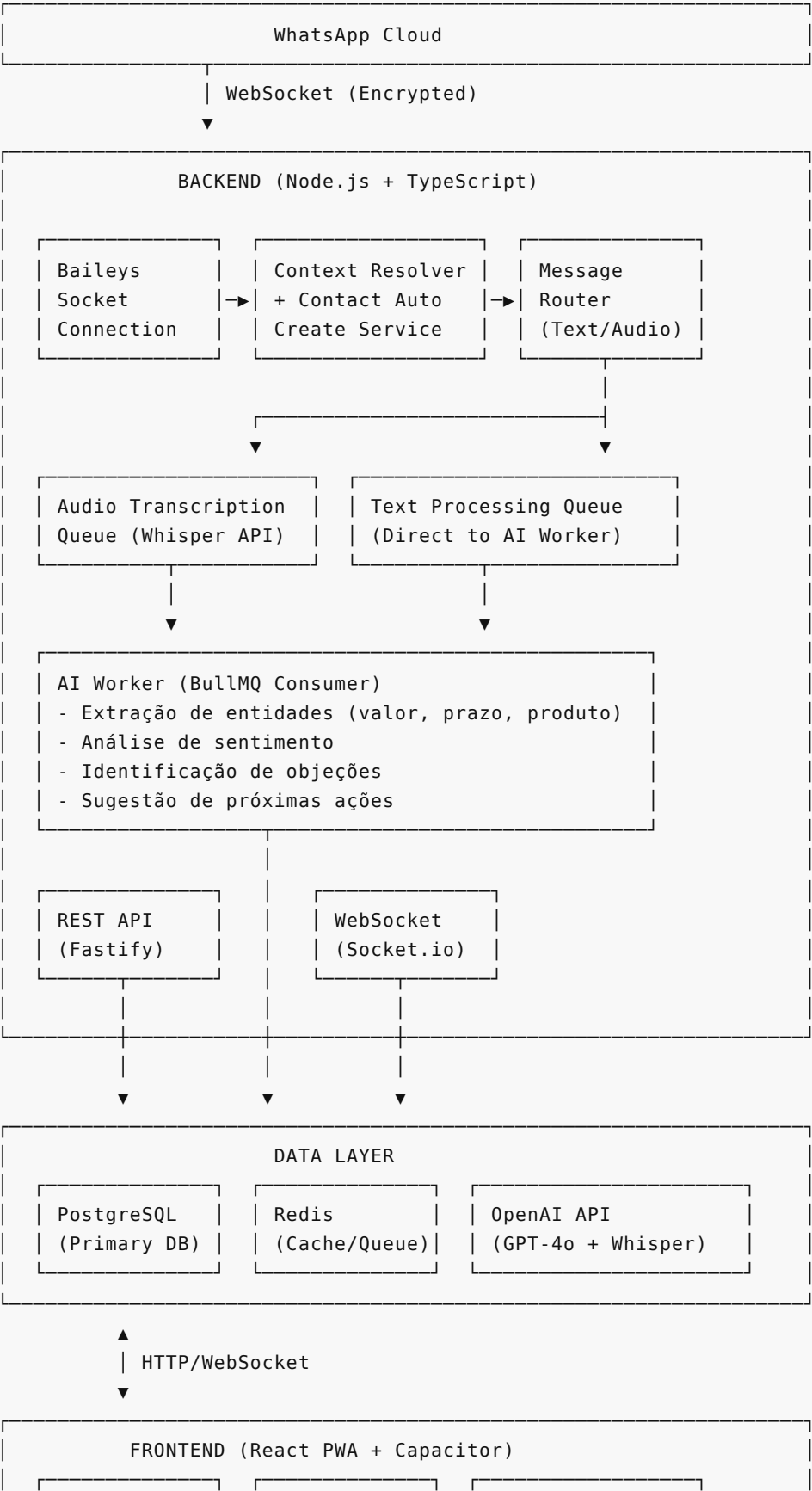
- Camada de Ingestão:** Responsável pela conexão WebSocket com WhatsApp via Baileys
- Camada de Processamento:** Workers assíncronos para análise de IA (texto e áudio)
- Camada de API:** Interface REST e WebSocket para o frontend
- Camada de Persistência:** Armazenamento estruturado e cache

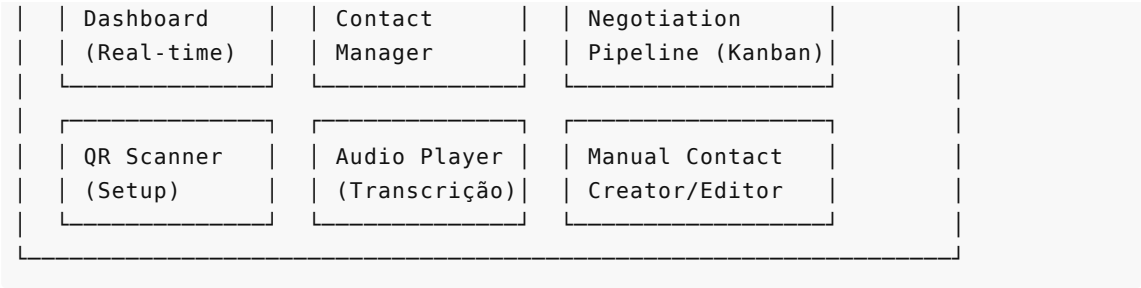
2.2 Modo Stealth

O Baileys opera como um cliente Web Multi-Device oficial do WhatsApp:

- Não depende do celular** estar ligado após autenticação inicial
- Não injeta scripts** no WhatsApp Web do navegador
- Estabelece sessão própria** via WebSocket criptografado
- Experiência transparente** para os contatos do Caio
- Única interação visível:** Escaneamento do QR Code na configuração inicial

2.3 Diagrama de Componentes





3. Stack Tecnológica Detalhada

3.1 Backend (Node.js + TypeScript)

Componente	Tecnologia	Versão Recomendada	Justificativa
Runtime	Node.js	20.x LTS	Performance I/O não bloqueante, essencial para WebSockets
Linguagem	TypeScript	5.x	Tipagem segura para dados complexos do WhatsApp
Framework HTTP	Fastify	4.x	Alta performance, schema validation nativo, plugins
ORM	Prisma	5.x	Type-safe queries, migrations, excelente DX
WebSocket Server	Socket.io	4.x	Real-time confiável com fallback para long polling
WhatsApp Client	Baileys	6.x (latest)	Multi-device, manutenção ativa, autenticação flexível
Queue Manager	BullMQ	5.x	Processamento robusto com Redis, retry policies, prioridades
Job Scheduler	node-cron	3.x	Tarefas agendadas (health checks, reconexão)

3.2 Banco de Dados

Tecnologia	Propósito	Estrutura
PostgreSQL 16+	Dados estruturados (CRM)	Contatos, Negociações, Mensagens, Análises IA, Transcrições
Redis 7+	Cache + Filas	Sessões Baileys, BullMQ, cache de consultas frequentes

3.3 Frontend (React)

Componente	Tecnologia	Justificativa
Framework	React 18+	Ecossistema maduro, componentização

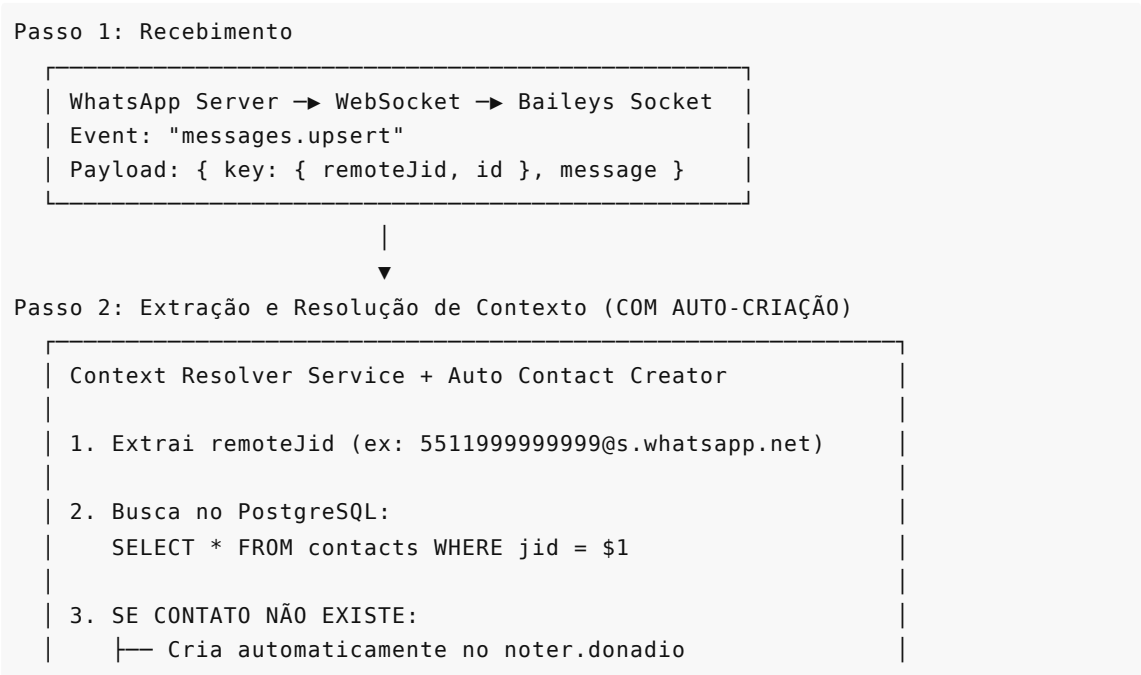
Build Tool	Vite 5+	Desenvolvimento rápido, HMR instantâneo
UI Framework	Tailwind CSS 3+	Estilização rápida, responsivo nativo
State Management	Zustand	Leve, sem boilerplate, compatível com TS
HTTP Client	Axios	Interceptors para tokens, retry automático
Real-time Client	Socket.io Client	Conexão bidirecional com backend
Mobile Wrapper	Capacitor 5+	Build nativo (iOS/Android) sem reescrever código
QR Scanner	html5-qrcode	Leitura do QR code de autenticação Baileys
Audio Player	Wavesurfer.js ou nativo	Reprodução de áudios originais e transcrições

3.4 IA/LLM

Provedor	Modelo Recomendado	Uso
OpenAI	GPT-4o-mini	Extração de entidades, análise de sentimento (texto)
OpenAI	GPT-4o	Casos complexos, negociações de alto valor
OpenAI	Whisper-1	Transcrição de mensagens de áudio
Estratégia	Prompt Engineering	Sem fine-tuning no MVP, prompts estruturados com JSON Mode

4. Fluxo de Dados Detalhado

4.1 Fluxo Principal: Ingestão de Mensagem (Texto)



```
| ┌─ INSERT INTO contacts (jid, phone_number,  
|   display_name, source, client_id)  
|   VALUES ($1, extractPhone($1),  
|   extractNameFromVCard($1) || 'Novo Contato',  
|   'whatsapp_auto', $clientId)  
| └─ Dispara evento "contact:created" via Socket.io  
|   └─ Log: "Novo contato criado automaticamente"
```

| 4. Busca negociação ativa:

```
| SELECT * FROM negotiations  
| WHERE contact_id = $1 AND status = 'active'
```

| 5. SE NÃO EXISTE NEGOCIAÇÃO ATIVA:

```
| ┌─ Cria lead automaticamente  
| └─ INSERT INTO negotiations (contact_id, status)  
|   VALUES ($1, 'lead')
```

|



Passo 3: Verificação do Tipo de Mensagem e Roteamento

```
| Message Router Service
```

```
| IF message.type === 'text':
```

```
|   └─ Enfileira direto em "ai-processing"
```

```
| IF message.type === 'audio' || 'ptt':
```

```
|   └─ Enfileira em "audio-transcription"
```

```
| IF message.type === 'image' || 'video':
```

```
|   └─ (Futuro) Enfileira em "media-analysis"
```

```
|   └─ MVP: Apenas registra metadados
```

|



Passo 4: Enfileiramento (Texto Direto)

```
| BullMQ Queue: "ai-processing"
```

```
| Job Data: {
```

```
|   messageId: "BAE5...",
```

```
|   jid: "55119...",
```

```
|   content: "Preciso de 50 unidades até sexta",
```

```
|   messageType: "text",
```

```
|   timestamp: "2026-07-20T14:30:00Z",
```

```
|   contactId: "uuid-123",
```

```
|   negotiationId: "uuid-456"
```

```
| }
```

|



Passo 5: Processamento IA (Worker)

```
| AI Worker (BullMQ Consumer)
| 1. Busca últimas 10 mensagens do contexto
| 2. Monta prompt estruturado:
|   System: "Você é um extrator de dados de
|   vendas para o noter.donadio.
|   Extraia: valor, prazo, produto,
|   quantidade, objeções, sentimento.
|   Formato JSON."
|   User: [últimas mensagens formatadas]
| 3. Envia para OpenAI API (JSON Mode)
| 4. Recebe resposta estruturada:
|   {
|     "summary": "...",
|     "entities": {
|       "value": 5000,
|       "deadline": "2026-07-25",
|       "product": "Widget X",
|       "quantity": 50
|     },
|     "sentiment": "positive",
|     "objections": ["prazo muito curto"],
|     "next_action": "Enviar proposta formal",
|     "should_update_negotiation": true
|   }
```

|



Passo 6: Persistência e Notificação

```
| Database Update:
| - INSERT INTO messages (id, content, ...)
| - INSERT INTO ai_analyses (...)
| - UPDATE negotiations SET
|   last_summary = $1,
|   sentiment = $2,
|   value = COALESCE($3, value),
|   updated_at = NOW()
|
| Event Emission (Socket.io):
| - Event: "negotiation:updated"
| - Room: "user:{clientId}"
| - Payload: negotiation object completo
```

|



Passo 7: Atualização Frontend

```
| React Dashboard:
| - Socket.io client recebe evento
| - Zustand store atualiza estado
| - Componente Kanban re-renderiza
| - Se novo contato: aparece na lista de contatos
```

```
| - Notificação toast (opcional)
```

4.2 Fluxo Secundário: Mensagem de Áudio (Transcrição)

Passo 1: Recebimento de Áudio

```
| WhatsApp Server → Baileys Socket  
| message.audioMessage || message.pttMessage  
| Contém: url (para download), seconds, mimetype
```



Passo 2: Download e Armazenamento

```
| Audio Handler Service  
| 1. Faz download do áudio via Baileys:  
|   const buffer = await downloadMediaMessage(  
|     message, 'buffer', {}  
|   );  
| 2. Salva em disco local ou S3:  
|   /audios/{messageId}.ogg  
| 3. Salva metadados no PostgreSQL:  
|   INSERT INTO audio_files (message_id,  
|     file_path, duration, mimetype)
```



Passo 3: Enfileiramento para Transcrição

```
| BullMQ Queue: "audio-transcription"  
| Job Data: {  
|   messageId: "BAE5...",  
|   jid: "55119...",  
|   audioPath: "/audios/BAE5.ogg",  
|   duration: 45,  
|   mimetype: "audio/ogg; codecs=opus",  
|   contactId: "uuid-123",  
|   negotiationId: "uuid-456"  
| }
```



Passo 4: Transcrição via Whisper API

```
| Transcription Worker  
| 1. Lê arquivo de áudio do disco/S3  
| 2. Envia para OpenAI Whisper API:  
|   const transcript = await openai.audio  
|     .transcriptions.create({  
|       model: "whisper-1",
```

```

    file: fs.createReadStream(audioPath),
    language: "pt", // Português
    response_format: "verbose_json"
  });
3. Recebe transcrição:
{
  text: "Preciso de 50 unidades do produto X
    entregues até sexta-feira",
  duration: 45.2,
  language: "portuguese",
  confidence: 0.95
}

```

|



Passo 5: Persistência e Redirecionamento

```

1. Salva transcrição no PostgreSQL:
UPDATE audio_files SET
  transcription_text = $1,
  transcription_confidence = $2,
  transcribed_at = NOW()

2. Cria uma mensagem "virtual" de texto:
INSERT INTO messages (
  contact_id, negotiation_id,
  content, direction,
  message_type, original_audio_message_id
) VALUES (
  $contactId, $negotiationId,
  $transcriptText, 'inbound',
  'transcription', $messageId
)

3. Dispara mesmo fluxo de texto:
Enfileira em "ai-processing"
com o texto transcrito

```

|



[Continua no Passo 5 do Fluxo de Texto]

4.3 Fluxo Terciário: Gerenciamento Manual de Contatos (Frontend)

Usuário (Caio) no Frontend

|



Tela de Contatos (Contact Manager)

Funcionalidades:

1. Listar todos os contatos (auto-criados + manuais) com filtros e busca
2. Criar contato manualmente:
 - Botão "Novo Contato"
 - Form: nome, telefone, tags, notas
 - POST /api/contacts
3. Editar contato (inclusive auto-criados):
 - Clique no contato abre drawer/modal
 - Campos editáveis: nome, foto, tags, notas internas, status
 - PUT /api/contacts/:id
4. Visualizar histórico:
 - Todas as mensagens do contato
 - Todas as negociações associadas
 - Transcrições de áudio com player
5. Mesclar contatos (opcional MVP):
 - Se detectar duplicata de telefone
 - Unificar histórico

5. Estratégias de Segurança

5.1 Autenticação e Sessão Baileys

Medida	Implementação
Criptografia de Credenciais	AES-256-GCM para arquivos de autenticação. Chave armazenada em variável de ambiente BAILEYS_ENCRYPTION_KEY
Armazenamento Seguro	Credenciais no PostgreSQL (tabela auth_sessions), nunca em disco não criptografado
Rotação de Chaves	Implementar endpoint para regenerar sessão se detectada atividade suspeita
Isolamento de Dados	Schema único para o cliente Caio Donadio. Preparado para multi-tenant futuro

5.2 Proteção de Dados

Camada	Medida
Transporte	TLS 1.3 em todas as conexões (API, WebSocket, DB)
Dados em Repouso	PostgreSQL com criptografia em nível de coluna para campos sensíveis
Anonimização de Logs	Nunca logar conteúdo de mensagens. Apenas IDs e metadados
Áudios	Armazenados com acesso restrito. URLs assinadas para playback

5.3 Proteção contra Abusos

Mecanismo	Descrição
Rate Limiting	Máximo 10 mensagens/minuto por contato (respeita limites do WhatsApp)
Circuit Breaker	Interrompe envios se detectar alta taxa de erros
Allowlist de Comandos	IA nunca envia mensagens diretamente no MVP. Envio sempre requer ação do Caio
Audit Trail	Toda ação de envio, criação/edição de contato registrada com timestamp

5.4 Segurança da Aplicação

Prática	Implementação
CORS	Configurado estritamente para domínios do frontend
Helmet.js	Headers de segurança HTTP (CSP, X-Frame-Options, etc.)
Input Validation	Zod schemas em todas as entradas da API
Dependency Scanning	npm audit + Dependabot no CI/CD
Secrets Management	Variáveis de ambiente injetadas via Docker secrets ou Vault

6. Modelagem de Dados (Schema Principal - noter.donadio)

6.1 PostgreSQL - Tabelas Core

```
-- Contatos (GERENCIÁVEIS VIA FRONTEND)
CREATE TABLE contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  jid VARCHAR(255) UNIQUE NOT NULL, -- ex: 551199999999@s.whatsapp.net
  phone_number VARCHAR(20) NOT NULL,
  display_name VARCHAR(255) DEFAULT 'Novo Contato',
  profile_picture_url TEXT,
  tags TEXT[], -- ex: {'cliente', 'fornecedor', 'vip'}
  notes TEXT, -- Notas internas do Caio
  source VARCHAR(50) DEFAULT 'whatsapp_auto', -- 'whatsapp_auto', 'manual',
'import'
  status VARCHAR(20) DEFAULT 'active', -- 'active', 'archived',
'blocked'
  metadata JSONB DEFAULT '{}', -- Dados extras flexíveis
  last_interaction_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  created_by VARCHAR(50) DEFAULT 'system' -- 'system' para auto-criados,
'caio' para manuais
);

-- Negociações (Pipeline)
```

```

CREATE TABLE negotiations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL REFERENCES contacts(id) ON DELETE CASCADE,
  title VARCHAR(255),
  status VARCHAR(50) DEFAULT 'lead',
  -- Status: 'lead', 'qualified', 'proposal_sent', 'in_negotiation',
  --         'closed_won', 'closed_lost', 'on_hold'
  value DECIMAL(15,2),
  currency VARCHAR(3) DEFAULT 'BRL',
  expected_close_date DATE,
  product_interest TEXT,
  last_summary TEXT,
  sentiment VARCHAR(20), -- 'positive', 'neutral', 'negative', 'urgent'
  priority INTEGER DEFAULT 0, -- 0-5 para ordenação no Kanban
  pipeline_stage_order INTEGER DEFAULT 0, -- Ordenação dentro do estágio
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Mensagens (Texto, Transcrições, Meta de Mídia)
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  whatsapp_message_id VARCHAR(255) UNIQUE,
  contact_id UUID NOT NULL REFERENCES contacts(id) ON DELETE CASCADE,
  negotiation_id UUID REFERENCES negotiations(id) ON DELETE SET NULL,
  direction VARCHAR(10) NOT NULL CHECK (direction IN ('inbound',
'outbound')),
  message_type VARCHAR(50) DEFAULT 'text'
    CHECK (message_type IN ('text', 'audio', 'image', 'video', 'document',
                             'transcription', 'system_note')),
  content TEXT, -- Texto original ou texto transcrito
  content_hash VARCHAR(64), -- SHA-256 para deduplicação
  media_url TEXT, -- URL para mídia original (áudio, imagem, etc.)
  media_mimetype VARCHAR(100),
  media_duration_seconds INTEGER,
  original_audio_message_id UUID REFERENCES messages(id),
  -- Se for transcrição, referencia a mensagem de áudio original
  timestamp TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Arquivos de Áudio (Metadados e Transcrições)
CREATE TABLE audio_files (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,
  file_path VARCHAR(500) NOT NULL, -- Caminho no disco/S3
  file_size_bytes BIGINT,
  duration_seconds INTEGER,
  mimetype VARCHAR(100),
  transcription_text TEXT,
  transcription_confidence DECIMAL(3,2),

```

```

transcription_language VARCHAR(10),
transcription_model VARCHAR(50) DEFAULT 'whisper-1',
transcribed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Análises da IA
CREATE TABLE ai_analyses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    message_id UUID REFERENCES messages(id) ON DELETE CASCADE,
    negotiation_id UUID REFERENCES negotiations(id) ON DELETE CASCADE,
    analysis_type VARCHAR(50) DEFAULT 'message_extraction',
    summary TEXT,
    entities JSONB, -- { value, deadline, product, quantity, ... }
    sentiment VARCHAR(20),
    sentiment_confidence DECIMAL(3,2),
    objections TEXT[],
    next_actions TEXT[],
    suggested_tags TEXT[], -- Tags sugeridas para o contato
    confidence_score DECIMAL(3,2),
    prompt_tokens INTEGER,
    completion_tokens INTEGER,
    model_used VARCHAR(100),
    processing_time_ms INTEGER,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Sessões de Autenticação (Baileys)
CREATE TABLE auth_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_identifier VARCHAR(100) UNIQUE NOT NULL DEFAULT 'caio_donadio',
    session_data BYTEA NOT NULL, -- Criptografado AES-256
    phone_number VARCHAR(20),
    is_active BOOLEAN DEFAULT true,
    qr_code_generated_at TIMESTAMPTZ,
    last_connected_at TIMESTAMPTZ,
    connection_status VARCHAR(20) DEFAULT 'disconnected',
    -- 'disconnected', 'qr_generated', 'connecting', 'connected', 'timeout'
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Índices para Performance
CREATE INDEX idx_contacts_jid ON contacts(jid);
CREATE INDEX idx_contacts_phone ON contacts(phone_number);
CREATE INDEX idx_contacts_source ON contacts(source);
CREATE INDEX idx_contacts_tags ON contacts USING GIN(tags);
CREATE INDEX idx_negotiations_contact ON negotiations(contact_id);
CREATE INDEX idx_negotiations_status ON negotiations(status);
CREATE INDEX idx_negotiations_priority ON negotiations(priority);
CREATE INDEX idx_messages_contact ON messages(contact_id);
CREATE INDEX idx_messages_negotiation ON messages(negotiation_id);

```

```
CREATE INDEX idx_messages_timestamp ON messages(timestamp);
CREATE INDEX idx_messages_type ON messages(message_type);
CREATE INDEX idx_audio_files_message ON audio_files(message_id);
CREATE INDEX idx_ai_analyses_negotiation ON ai_analyses(negotiation_id);
CREATE INDEX idx_ai_analyses_message ON ai_analyses(message_id);
```

6.2 Redis - Estruturas

```
# Cache de Sessão Baileys (Temporário)
KEY: "baileys:auth:caio_donadio"
VALUE: JSON criptografado
TTL: 24 horas

# Fila: Processamento de Texto
QUEUE: "ai-processing"
  - delayed (mensagens agendadas)
  - waiting (aguardando processamento)
  - active (em processamento)
  - completed (últimas 1000)
  - failed (com retry policy: 3 tentativas)

# Fila: Transcrição de Áudio
QUEUE: "audio-transcription"
  - waiting (áudios aguardando transcrição)
  - active (em processamento Whisper)
  - completed (últimas 500)
  - failed (com retry policy: 2 tentativas)

# Cache de Contatos Recentes
KEY: "contacts:recent:caio_donadio"
VALUE: Sorted Set (timestamp score)
TTL: 1 hora

# Cache de Negociações Ativas
KEY: "negotiations:active:caio_donadio"
VALUE: JSON (últimos dados do pipeline)
TTL: 5 minutos

# Rate Limiting por Contato
KEY: "ratelimit:send:{jid}"
VALUE: Counter
TTL: 1 minuto (sliding window)

# Lock para evitar duplicação de processamento
```

KEY: "lock:message:{whatsapp_message_id}"
VALUE: "1"
TTL: 60 segundos

7. Estrutura do Projeto

```
noter-donadio/  
├─ backend/  
│   └─ src/  
│       ├── config/  
│       │   ├── database.ts          # Prisma client  
│       │   ├── redis.ts             # Redis connection  
│       │   └─ env.ts                # Environment variables (Zod)  
│       └─ modules/  
│           ├── whatsapp/  
│           │   ├── baileys.service.ts # Socket connection, auth  
│           │   ├── message.handler.ts # Event listeners + router  
│           │   ├── sender.service.ts  # Send messages  
│           │   └─ audio.handler.ts    # Download + transcode  
│           ├── contacts/  
│           │   ├── contacts.controller.ts # CRUD endpoints  
│           │   ├── contacts.service.ts    # Business logic  
│           │   └─ auto-creator.service.ts # Criação automática  
│           ├── negotiations/  
│           │   ├── negotiations.controller.ts  
│           │   └─ negotiations.service.ts  
│           ├── ai/  
│           │   ├── text.worker.ts        # BullMQ: ai-processing  
│           │   ├── transcription.worker.ts # BullMQ: audio-transcription  
│           │   ├── prompts/  
│           │   │   ├── extraction.ts      # Extração de entidades  
│           │   │   └─ contact-enrichment.ts # Sugestão de tags/nome  
│           │   ├── llm.service.ts         # OpenAI (GPT-4o)  
│           │   └─ whisper.service.ts      # OpenAI (Whisper)  
│           └─ websocket/  
│               └─ socket.gateway.ts       # Socket.io handlers  
│       ├── middleware/  
│       │   ├── auth.middleware.ts  
│       │   ├── ratelimit.middleware.ts  
│       │   └─ validation.middleware.ts  
│       ├── queue/  
│       │   ├── queue.config.ts           # BullMQ setup  
│       │   └─ bull-board.ts              # Dashboard de monitoramento  
│       └─ app.ts                         # Fastify app setup  
├─ prisma/  
│   ├── schema.prisma  
│   └─ migrations/  
├─ storage/  
│   └─ audios/                          # Áudios baixados (gitignored)  
└─ Dockerfile
```

```
|   ├── Dockerfile.worker
|   └── package.json
├── frontend/
|   ├── src/
|   |   ├── components/
|   |   |   ├── Dashboard/
|   |   |   │   └── StatsCards.tsx
|   |   |   ├── KanbanBoard/
|   |   |   │   ├── KanbanColumn.tsx
|   |   |   │   └── NegotiationCard.tsx
|   |   |   ├── Contacts/
|   |   |   │   ├── ContactList.tsx
|   |   |   │   ├── ContactDetail.tsx
|   |   |   │   ├── ContactForm.tsx          # Criar/Editar
|   |   |   │   └── ContactMerge.tsx        # Mesclar duplicatas
|   |   |   ├── ChatView/
|   |   |   │   ├── MessageList.tsx
|   |   |   │   ├── MessageBubble.tsx
|   |   |   │   └── AudioPlayer.tsx          # Player de áudio
|   |   |   ├── QRScanner/
|   |   |   │   └── QRSetup.tsx
|   |   |   └── common/
|   |   |       ├── Loading.tsx
|   |   |       ├── EmptyState.tsx
|   |   |       └── Toast.tsx
|   |   ├── hooks/
|   |   |   ├── useSocket.ts
|   |   |   ├── useApi.ts
|   |   |   └── useContacts.ts
|   |   ├── store/
|   |   |   ├── contactsStore.ts
|   |   |   ├── negotiationsStore.ts
|   |   |   └── uiStore.ts
|   |   ├── services/
|   |   |   ├── api.ts                      # Axios instance
|   |   └── pages/
|   |       ├── DashboardPage.tsx
|   |       ├── ContactsPage.tsx
|   |       ├── NegotiationDetailPage.tsx
|   |       └── SetupPage.tsx
|   |   ├── App.tsx
|   |   └── main.tsx
|   ├── public/
|   ├── capacitor.config.ts
|   ├── Dockerfile
|   └── package.json
├── docker-compose.yml
├── .env.example
├── .gitignore
└── README.md
```

8. Estratégia de Desenvolvimento (Ordem de Execução MVP)

Para validar o mais rápido possível e garantir que o frontend seja totalmente funcional, siga esta ordem de execução faseada:

Fase 0: Fundação e Frontend Estático (Dia 1-2)

Objetivo: Ver o frontend completo, mockando os dados.

Tarefas:

1. Inicializar monorepo (backend + frontend)
2. Configurar TypeScript, ESLint, Prettier
3. Backend: Configurar Fastify, Prisma, criar schema completo
4. Frontend: Criar todas as telas com React + Tailwind:
 - Dashboard com cards de estatísticas
 - Kanban board com colunas de pipeline
 - Lista de contatos com busca e filtros
 - Formulário de criação/edição de contato
 - Tela de detalhes da negociação com histórico de mensagens
 - Componente AudioPlayer para transcrições
 - Tela de setup/QR code
5. Mockar API com MSW (Mock Service Worker) ou JSON Server
6. **Resultado:** Frontend 100% navegável com dados fake. Caio já pode ver e aprovar o layout.

Fase 1: API REST e Banco Real (Dia 3-4)

Objetivo: Substituir mocks por API real, sem WhatsApp ainda.

Tarefas:

1. Rodar migrations e criar banco PostgreSQL
2. Implementar CRUD completo:
 - `GET/POST /api/contacts` (listar e criar contatos)
 - `GET/PUT /api/contacts/:id` (detalhe e edição)
 - `GET /api/negotiations` (pipeline completo)
 - `GET/PATCH /api/negotiations/:id` (detalhe e atualização manual)
 - `GET /api/messages?contactId=X` (histórico)
3. Implementar Socket.io para real-time (por enquanto, eventos manuais)
4. Criar endpoint `POST /api/mock/message` para simular chegada de mensagem
5. **Resultado:** Frontend conectado ao backend real. Caio pode criar/editar contatos manualmente.

Fase 2: Conexão WhatsApp Real (Dia 5-6)

Objetivo: O sistema lê mensagens reais do WhatsApp.

Tarefas:

1. Implementar Baileys com `makeWASocket` e `useMultiFileAuthState`
2. Tela de QR Code no frontend funcional
3. Handler de `messages.upsert` com:
 - Auto-criação de contato se não existir
 - Criação automática de lead/negociação

- Roteamento: texto → fila de IA, áudio → fila de transcrição
- 4. Implementar download de áudios do WhatsApp
- 5. Salvar mensagens no banco
- 6. **Resultado:** Mensagens reais aparecendo no frontend, contatos sendo criados automaticamente.

Fase 3: IA e Transcrição (Dia 7-8)

Objetivo: A IA processa mensagens e áudios.

Tarefas:

1. Configurar BullMQ com Redis (filas: `ai-processing` e `audio-transcription`)
2. Implementar Text Worker:
 - Prompt de extração de entidades
 - Integração com OpenAI GPT-4o-mini
 - Salvar análises no banco
 - Emitir evento Socket.io para atualizar frontend
3. Implementar Transcription Worker:
 - Download do arquivo de áudio
 - Envio para Whisper API
 - Salvar transcrição
 - Criar mensagem virtual e enfileirar para IA
4. Testar ponta a ponta:
 - Enviar áudio pelo WhatsApp → ver transcrição no frontend → ver análise no Kanban
5. **Resultado:** Pipeline completo funcionando. Áudios transcritos e analisados.

Fase 4: Ações e Polimento (Dia 9-10)

Objetivo: Fechar o ciclo e refinar experiência.

Tarefas:

1. Implementar envio de mensagens via frontend (ação do Caio)
2. Audio Player funcional com transcrição sincronizada (opcional: timestamps)
3. Mesclagem de contatos duplicados (mesmo telefone)
4. Filtros avançados no Kanban (por sentimento, valor, data)
5. Notificações toast para:
 - Novo contato criado automaticamente
 - Nova negociação detectada
 - Transcrição concluída
 - Análise de IA finalizada
6. Responsividade mobile total
7. **Resultado:** MVP completo e funcional.

Fase 5: Deploy e Mobile (Dia 11-12)

Objetivo: Sistema em produção, acessível como app.

Tarefas:

1. Configurar Docker Compose para produção
2. Adicionar Capacitor ao React
3. Gerar build mobile (APK para Android, opcional iOS)
4. Configurar Nginx como proxy reverso

5. Deploy em VPS (DigitalOcean/Hetzner)
 6. Configurar domínio e HTTPS (Let's Encrypt)
 7. Configurar backup automático do banco
 8. Testes finais de stress e usabilidade
 9. **Resultado:** Sistema em produção, Caio acessa via navegador e app mobile.
-

9. Sugestões de Deploy (MVP)

9.1 Infraestrutura Recomendada

Opção 1: VPS Única (Recomendado para MVP)

- **Provedor:** DigitalOcean, AWS Lightsail, ou Hetzner
- **Especificações:** 4GB RAM, 2 vCPUs, 80GB SSD
- **Sistema:** Ubuntu 22.04 LTS
- **Orquestração:** Docker Compose
- **Domínio:** noter.donadio (ou subdomínio)

Opção 2: Serverless + Managed Services (Escalável)

- **API:** AWS ECS Fargate ou Railway
- **Banco:** AWS RDS PostgreSQL + ElastiCache Redis
- **Workers:** AWS ECS Tasks separadas
- **Frontend:** Vercel ou Cloudflare Pages
- **Áudios:** AWS S3 com CloudFront

9.2 Docker Compose (Produção)

```
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    container_name: noter-postgres
    environment:
      POSTGRES_DB: noter_donadio
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./backend/prisma/migrations:/docker-entrypoint-initdb.d
    networks:
      - noter_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d noter_donadio"]
      interval: 10s
      timeout: 5s
      retries: 5
      restart: unless-stopped

  redis:
    image: redis:7-alpine
```

```

    container_name: noter-redis
    command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD} --
maxmemory 256mb --maxmemory-policy allkeys-lru
    volumes:
      - redis_data:/data
    networks:
      - noter_network
    healthcheck:
      test: ["CMD", "redis-cli", "--pass", "${REDIS_PASSWORD}", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: noter-backend
  environment:
    NODE_ENV: production
    DATABASE_URL:
postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/noter_donadio
    REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
    OPENAI_API_KEY: ${OPENAI_API_KEY}
    BAILEYS_ENCRYPTION_KEY: ${BAILEYS_ENCRYPTION_KEY}
    JWT_SECRET: ${JWT_SECRET}
    AUDIO_STORAGE_PATH: /app/storage/audios
  volumes:
    - audio_storage:/app/storage/audios
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  networks:
    - noter_network
  restart: unless-stopped

worker-text:
  build:
    context: ./backend
    dockerfile: Dockerfile.worker
  container_name: noter-worker-text
  command: node dist/modules/ai/text.worker.js
  environment:
    NODE_ENV: production
    DATABASE_URL:
postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/noter_donadio
    REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
    OPENAI_API_KEY: ${OPENAI_API_KEY}
  depends_on:

```

```

    - postgres
    - redis
networks:
    - noter_network
restart: unless-stopped
deploy:
    replicas: 2

worker-transcription:
    build:
        context: ./backend
        dockerfile: Dockerfile.worker
    container_name: noter-worker-transcription
    command: node dist/modules/ai/transcription.worker.js
    environment:
        NODE_ENV: production
        DATABASE_URL:
postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/noter_donadio
        REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
        OPENAI_API_KEY: ${OPENAI_API_KEY}
        AUDIO_STORAGE_PATH: /app/storage/audios
    volumes:
        - audio_storage:/app/storage/audios
    depends_on:
        - postgres
        - redis
    networks:
        - noter_network
    restart: unless-stopped

nginx:
    image: nginx:alpine
    container_name: noter-nginx
    ports:
        - "80:80"
        - "443:443"
    volumes:
        - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
        - ./nginx/ssl:/etc/nginx/ssl:ro
        - frontend_build:/usr/share/nginx/html:ro
    depends_on:
        - backend
    networks:
        - noter_network
    restart: unless-stopped

# Opcional: Bull Board para monitorar filas
bullboard:
    image: felixmosh/bull-board
    container_name: noter-bullboard
    ports:
        - "3001:3000"

```

```
environment:
  REDIS_HOST: redis
  REDIS_PORT: 6379
  REDIS_PASSWORD: ${REDIS_PASSWORD}
  BULL_PREFIX: bull
depends_on:
  - redis
networks:
  - noter_network
restart: unless-stopped

networks:
  noter_network:
    driver: bridge

volumes:
  postgres_data:
  redis_data:
  audio_storage:
  frontend_build:
```

9.3 CI/CD Pipeline (GitHub Actions)

```
name: Deploy noter.donadio

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_DB: noter_donadio_test
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
        ports:
          - 5432:5432
      redis:
        image: redis:7-alpine
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v4
```

```

- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm'

- name: Install Backend Dependencies
  run: | cd backend
    npm ci

- name: Run Backend Tests
  run: | cd backend
    npm run test
  env:
    DATABASE_URL:
postgresql://test:test@localhost:5432/noter_donadio_test
    REDIS_URL: redis://localhost:6379

- name: Install Frontend Dependencies
  run: | cd frontend
    npm ci

- name: Run Frontend Tests
  run: | cd frontend
    npm run test

- name: Build Frontend
  run: | cd frontend
    npm run build

deploy:
  needs: test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: Deploy to VPS
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.VPS_HOST }
        username: ${ secrets.VPS_USER }
        key: ${ secrets.SSH_PRIVATE_KEY }
        script: | cd /opt/noter-donadio
          git pull origin main
          docker compose down
          docker compose build --no-cache
          docker compose up -d
          docker system prune -f
          echo "Deploy concluído em $(date)"

```

10. Considerações Técnicas Importantes

10.1 Auto-Criação de Contatos (Regras de Negócio)

- **Quando criar:** Toda mensagem recebida de um JID não cadastrado
- **Dados iniciais:**
 - `phone_number` : Extraído do JID
 - `display_name` : Do vCard do WhatsApp (se disponível) ou "Novo Contato + DDD"
 - `source` : "whatsapp_auto"
 - `tags` : Array vazio (Caio pode adicionar depois)
- **Notificação:** Toast no frontend "Novo contato criado: +55 11 99999-9999"
- **Edição:** Imediatamente editável via frontend (nome, tags, notas)
- **Auditoria:** Log de criação automática para evitar duplicatas indesejadas

10.2 Transcrição de Áudio (Decisões Técnicas)

- **Formato:** WhatsApp envia áudio em OGG/Opus. Whisper aceita vários formatos (mp3, mp4, mpeg, mpga, m4a, wav, webm)
- **Estratégia:**
 - Opção A (MVP): Baixar OGG e enviar direto para Whisper (whisper-1 aceita ogg)
 - Opção B (Otimizado): Converter para WAV 16kHz mono com FFmpeg antes de enviar (menor latência)
- **Custo:** Whisper-1 custa \$0.006/minuto. Áudio médio de 30s = \$0.003 por transcrição
- **Idioma:** Forçar `language`: "pt" para melhor acurácia em português
- **Filas separadas:** Transcrição pode demorar (1-3 segundos por minuto de áudio). Manter em fila separada evita bloquear processamento de texto

10.3 Limitações do Baileys

- **Conexão Única por Número:** Apenas uma instância por número de WhatsApp
- **Reconexão Automática:** Implementar lógica de retry com backoff exponencial
- **Bloqueios:** WhatsApp pode bloquear temporariamente por comportamento suspeito
- **Rate Limiting:** Respeitar pausas entre mensagens (mínimo 1-2 segundos)
- **Áudios longos:** Podem levar tempo para download. Implementar timeout de 60 segundos

10.4 Otimizações de Custo IA

Recurso	Estratégia	Economia Estimada
GPT-4o-mini	Usar como padrão para extração	90% mais barato que GPT-4o
GPT-4o	Reservar para casos complexos (valor alto)	Uso seletivo
Cache de Prompts	System prompt fixo, variar apenas user	Menos tokens processados
Batch Processing	Processar mensagens em lotes noturnos	Menos chamadas concorrentes
Whisper	Apenas transcrever áudios < 5 minutos	Evitar custos com áudios longos

10.5 Escalabilidade Futura

Camada	Estratégia Atual (MVP)	Estratégia Futura
API	Single container	Horizontal scaling com load balancer
Workers	2 text + 1 transcription	Aumentar réplicas conforme volume

Banco	PostgreSQL single node	Read replicas, particionamento
WebSocket	Socket.io single instance	Redis adapter para multi-instância
Áudios	Disco local no container	AWS S3 com CloudFront CDN
Baileys	1 número WhatsApp	Arquitetura multi-instância para múltiplos números

11. Funcionalidades do Frontend (Visão do Usuário - Caio)

11.1 Tela de Dashboard

- Cards com métricas: Total de contatos, negociações ativas, valor total em pipeline, áudios pendentes
- Gráfico de negociações por status (lead, qualificado, proposta, etc.)
- Timeline de últimas atividades (mensagens recebidas, contatos criados, transcrições concluídas)
- Indicador de status do WhatsApp (conectado/desconectado)

11.2 Tela de Contatos (Contact Manager)

- **Lista:** Tabela com colunas: nome, telefone, tags, última interação, status
- **Filtros:** Por tag, status, data de criação, fonte (auto/manual)
- **Busca:** Por nome ou telefone (busca em tempo real)
- **Ações:**
 - Criar novo contato manualmente (botão "+ Novo Contato")
 - Editar contato (inclusive auto-criados): nome, tags, notas, foto
 - Arquivar/Desarquivar contato
 - Ver detalhes do contato (histórico de mensagens, negociações)
 - Mesclar contatos duplicados (se detectado mesmo telefone)

11.3 Tela de Pipeline/Kanban

- **Colunas:** Lead, Qualificado, Proposta Enviada, Em Negociação, Fechado (Ganho/Perdido)
- **Cards:** Cada negociação mostra:
 - Nome do contato
 - Último resumo (IA)
 - Valor (se detectado)
 - Sentimento (emoji + cor: verde/amarelo/vermelho)
 - Data da última interação
 - Indicador se tem áudio transcrito pendente
- **Drag & Drop:** Mover cards entre colunas (atualiza status)
- **Ordenação:** Por prioridade, valor, data (configurável)
- **Filtros:** Por sentimento, valor estimado, data prevista de fechamento

11.4 Tela de Detalhes da Negociação

- **Header:** Nome do contato, status atual, valor, botão de editar
- **Timeline de Mensagens:**
 - Mensagens de texto (com análise da IA expandível)
 - Mensagens de áudio com player e transcrição
 - Indicador visual do que foi transcrito vs. aguardando
- **Resumo da IA:** Card destacado com:
 - Resumo da última análise
 - Entidades extraídas (valor, produto, prazo, quantidade)

- Sentimento detectado
- Objeções identificadas
- Próximas ações sugeridas
- **Caixa de Resposta:** Enviar mensagem de texto para o contato (ação manual do Caio)
- **Ações Manuais:**
 - Atualizar status da negociação
 - Adicionar nota interna
 - Alterar valor manualmente
 - Arquivar negociação

11.5 Player de Áudio (Componente)

- Barra de progresso com waveform (usando Wavesurfer.js ou nativo)
- Controles: play/pause, velocidade (1x, 1.5x, 2x), volume
- Transcrição sincronizada (opcional MVP): destaca texto conforme áudio toca
- Download do áudio original
- Indicador de status: "Transcrevendo...", "Transcrição concluída", "Falha na transcrição"

11.6 Tela de Setup/QR Code

- Instruções passo a passo para conectar WhatsApp
- Leitor de QR Code usando câmera (html5-qrcode)
- Status da conexão em tempo real
- Botão para reconectar/regenerar QR code
- Indicador de saúde do sistema (ping, banco de dados, Redis, OpenAI)

12. Plano de Contingência

12.1 Falhas Comuns e Soluções

Problema	Sintoma	Solução Imediata	Prevenção
Desconexão WhatsApp	Frontend mostra "Desconectado"	Reconexão automática com backoff exponencial (5s, 10s, 30s, 1min, 5min)	Healthcheck a cada 30s, alerta se offline > 5min
API OpenAI fora do ar	Filas acumulando, análises paradas	Circuit breaker (para após 5 falhas), notificar Caio	Fallback para Claude API, cache de prompts
Whisper API lenta/indisponível	Áudios acumulando na fila	Pausar transcrições, priorizar texto. Notificar: "Transcrições temporariamente pausadas"	Timeout de 30s, retry 2x, marcar áudio como "pending_retry"
Fila acumulando	Dashboard mostra processamento lento	Escalar workers temporariamente (docker compose up -d --scale worker-text=4)	Alertas no Bull Board, métricas de fila no dashboard
Sessão Baileys expirada	Mensagens não chegam	Regenerar QR code automaticamente, notificar Caio para re-escanear	Rotação proativa a cada 30 dias, backup da sessão criptografada

Disco cheio (áudios)	Erro ao baixar novos áudios	Limpeza automática de áudios > 30 dias	Monitoramento de disco, política de retenção configurável
Duplicata de processamento	Mesma mensagem processada 2x	Lock distribuído no Redis: lock:message:{id} com TTL 60s	Verificar content_hash antes de processar

- **PostgreSQL:**
 - Backup automático diário (pg_dump custom format)
 - WAL archiving para Point-in-Time Recovery
 - Retenção: 7 backups diários + 4 semanais
- **Redis:**
 - Append-only file (AOF) para durabilidade
 - Snapshot RDB a cada 6 horas
- **Sessões Baileys:**
 - Backup criptografado diário da tabela auth_sessions
 - Cópia para bucket S3 (ou outro storage externo)
- **Áudios:**
 - Backup incremental semanal para S3
 - Política de lifecycle: manter 90 dias
- **RTO (Recovery Time Objective):** < 1 hora para MVP
- **RPO (Recovery Point Objective):** < 5 minutos (com WAL shipping)

12.3 Monitoramento Recomendado

```
# Script de healthcheck (cron a cada 1 minuto)
#!/bin/bash
curl -f http://localhost:3000/health || \
  docker compose restart backend worker-text worker-transcription
```

- **Endpoint /health** : Retorna status de DB, Redis, OpenAI, Baileys
- **Prometheus + Grafana (futuro):** Métricas detalhadas de filas, latência, uso de memória
- **Bull Board:** Dashboard visual das filas (acessível em :3001)
- **Logs:** Centralizados com Pino, exportáveis para ELK/Sentry no futuro

13. Apêndice: Variáveis de Ambiente

```
# .env.example (NUNCA COMMITAR O .env REAL)

# noter.donadio - Environment Variables

# ===== DATABASE =====
DATABASE_URL=postgresql://noter_user:strong_password_here@postgres:5432/noter_db
DB_USER=noter_user
```

```
DB_PASSWORD=strong_password_here
```

```
# ===== REDIS =====
```

```
REDIS_URL=redis://:redis_password_here@redis:6379
```

```
REDIS_PASSWORD=redis_password_here
```

```
# ===== OPENAI =====
```

```
OPENAI_API_KEY=sk-proj-...
```

```
OPENAI_MODEL=gpt-4o-mini # Padrão para extração
```

```
OPENAI_MODEL_PREMIUM=gpt-4o # Para casos complexos (valor alto)
```

```
OPENAI_MAX_TOKENS=1000
```

```
OPENAI_TEMPERATURE=0.1 # Baixa para extração consistente
```

```
# ===== WHISPER (Transcrição) =====
```

```
WHISPER_MODEL=whisper-1
```

```
WHISPER_LANGUAGE=pt # Forçar português
```

```
WHISPER_TIMEOUT_MS=30000 # 30 segundos
```

```
# ===== BAILEYS (WhatsApp) =====
```

```
BAILEYS_ENCRYPTION_KEY=base64-encoded-32-byte-key-here
```

```
BAILEYS_RECONNECT_INTERVAL=5000 # 5 segundos iniciais
```

```
BAILEYS_MAX_RETRIES=10
```

```
# ===== JWT (Frontend Auth) =====
```

```
JWT_SECRET=random-secret-string-at-least-32-chars
```

```
JWT_EXPIRATION=7d
```

```
# ===== APP =====
```

```
NODE_ENV=production
```

```
PORT=3000
```

```
HOST=0.0.0.0
```

```
LOG_LEVEL=info
```

```
CLIENT_NAME="Caio Donadio"
```

```
CLIENT_IDENTIFIER=caio_donadio
```

```
# ===== RATE LIMITING =====
```

```
RATE_LIMIT_MAX=100
```

```
RATE_LIMIT_WINDOW_MS=60000 # 1 minuto
```

```
WHATSAPP_MESSAGE_DELAY_MS=1500 # 1.5 segundos entre mensagens
```

```
# ===== STORAGE =====
```

```
AUDIO_STORAGE_PATH=/app/storage/audios
```

```
AUDIO_MAX_DURATION_SECONDS=300 # 5 minutos máximo
```

```
AUDIO_RETENTION_DAYS=90
```

```
# ===== BULLMQ =====
BULLMQ_CONCURRENCY_TEXT=5 # Processar 5 textos simultaneamente
BULLMQ_CONCURRENCY_TRANSCRIPTION=2 # Processar 2 transcrições simultaneamente
BULLMQ_RETRY_MAX=3
BULLMQ_RETRY_DELAY=1000 # 1 segundo entre retries

# ===== MONITORING =====
HEALTH_CHECK_INTERVAL_MS=30000 # 30 segundos
ENABLE_BULL_BOARD=true
BULL_BOARD_PORT=3001
```

14. Conclusão

Esta arquitetura fornece uma base sólida para o MVP do **noter.donadio**, o assistente corporativo stealth desenvolvido pela **Ruas.dev** para **Caio Donadio**.

Diferenciais implementados conforme solicitado:

1. **✓ Auto-criação de contatos:** Todo remetente desconhecido é automaticamente cadastrado no noter.donadio, com notificação em tempo real e total gerenciabilidade via frontend
2. **✓ Pipeline de transcrição de áudio:** Áudios são baixados, transcritos via Whisper API, e o texto resultante segue o mesmo fluxo de análise de IA que mensagens de texto
3. **✓ Gerenciamento completo de contatos:** Frontend permite criar, editar, taggear, mesclar e gerenciar tanto contatos auto-criados quanto manuais
4. **✓ Player de áudio com transcrição:** Caio pode ouvir os áudios originais e ler as transcrições diretamente no frontend

Pontos fortes da solução:

- **Baixa latência percebida:** Processamento assíncrono via filas separadas (texto e áudio)
- **Resiliência:** Retry policies, circuit breakers, health checks, locks anti-duplicação
- **Segurança:** Criptografia AES-256 para sessões, TLS 1.3, anonimização de logs
- **Experiência mobile nativa:** React PWA + Capacitor, sem reescrever código
- **Custo controlado:** GPT-4o-mini como padrão, Whisper apenas para áudios, cache de prompts
- **Estratégia faseada:** Frontend funcional desde o Dia 2, permitindo validação visual rápida por Caio

O sistema está pronto para ser implementado seguindo a estratégia de desenvolvimento faseada (5 fases, ~12 dias), permitindo validação rápida, feedback contínuo, e iterações ágeis.

Documento preparado por: Ruas.dev - Irving G. Ruas Lopes

Para: Caio Donadio

Projeto: noter.donadio - Pipeline Inteligente de Contatos e Negociações

Classificação: Confidencial

Próxima revisão: Após feedback do MVP inicial (Fase 1)

"Transformando conversas em negócios, silenciosamente." Ruas.dev - Engenharia de Software de Alta Performance